

Capitolo 11 — INDEX-FIRST spiegato passo per passo

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-11
Data master	2026-08-29
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/11_index_first_spiegato_passo_per_passo.md
Source SHA	87b1a31
Release	2026-08-29T07:37:59.861Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

Stato: FROZEN **Parte:** IV — INDEX-FIRST: come WCM trova quello che gli serve **Scope:** WCM generale / domain-agnostic **Review:** Technical Review PASS / Human Comprehension Review PASS

11.0 Dalla mappa al movimento

Nel Capitolo 10 abbiamo visto il Knowledge Navigation Layer.

Ora lo usiamo.

INDEX-FIRST non significa semplicemente:

! "apri prima un file chiamato index".

È una disciplina di retrieval.

Il suo obiettivo è trasformare una domanda in un percorso di lettura progressivo, proporzionato e verificabile.

```
L0 — ENTRY POINT
↓
L1 — MAP / INDEX
↓
L2 — AUTHORITY / PROCEDURE
↓ solo se necessario
L3 — EVIDENCE / HISTORY / RAW
↓
STOP WHEN SUFFICIENT
```

La cosa più importante da capire è questa:

! **non è obbligatorio arrivare fino a L3.**

Ogni passaggio deve essere giustificato da un'informazione che manca, da una contraddizione o da una necessità di verifica.

11.1 Prima domanda: che cosa sto cercando?

Prima ancora di aprire un indice, bisogna capire il task.

Una richiesta vaga produce un retrieval vago.

Esempio astratto:

| *"Controlla questa cosa."*

Prima di cercare file, dobbiamo tradurre la richiesta in una domanda operativa.

Potrebbe voler dire:

- verificare uno stato;
- trovare una regola;
- capire una decisione;
- eseguire un processo;
- ricostruire una storia;
- controllare una contraddizione.

INDEX-FIRST parte quindi da una domanda semplice:

| **qual è l'informazione che mi serve per poter agire correttamente?**

11.2 L0 — Entry Point

L0 è il livello di orientamento.

Nel WCM generale il punto di ingresso corrente è WCM_AGENT_START.md.

Il suo ruolo è ridurre immediatamente lo spazio di ricerca.

L0 deve aiutare a capire:

- chi è l'attore;
- qual è il task o goal;
- se esiste Working Memory pertinente;
- se esiste un workflow già aperto da riprendere;
- quale area del WCM è coinvolta;
- quale entry point specifico o indice aprire.

Il risultato corretto di L0 non è:

| *"ho capito tutto".*

È:

| **"so da dove devo entrare."**

11.3 L0 e Working Memory

INDEX-FIRST non ignora ciò che il sistema sa già.

Se la Working Memory contiene contesto recente e affidabile, può essere usato.

CONTESTO GIÀ NOTO
+
NUOVO TASK

↓
QUALI INFORMAZIONI MANCANO DAVVERO?

Questo evita il comportamento:

NUOVA DOMANDA
→ DIMENTICA TUTTO
→ RICOSTRUISCI DA ZERO

Ma resta valida una regola fondamentale:

| Memory is not authority.

La Working Memory può orientare.

Non sostituisce una verifica persistente quando il task richiede status, authority o baseline corrente.

11.4 L0 e Resume Priority

Prima di cercare nuovo lavoro, PROC-005 richiede di verificare se esiste un workflow da riprendere.

Se troviamo ACTIVE con true stop non raggiunta oppure INTERRUPTED_RESUMABLE, la route corretta può essere:

RESUME

non:

DISCOVER NEW TASK

Questo è INDEX-FIRST applicato alla continuità operativa.

La prima cosa da recuperare non è sempre un documento.

A volte è il **percorso già in corso**.

11.5 L0: quando basta

In alcuni task molto semplici L0 può già essere sufficiente.

Per esempio:

| *"Dove trovo il Process Book?"*

L'Entry Point può già indicare la route.

Non serve aprire processi, protocolli, evidence e storico.

Questa è la prima applicazione concreta di:

| Stop When Sufficient.

11.6 L1 — Map / Index

Se L0 non basta, si passa a L1.

L1 non serve ancora a leggere il contenuto profondo.

Serve a identificare le fonti candidate.

Un buon indice permette di capire:

- quali nodi esistono;
- quale tipo di nodo rappresentano;
- quale status hanno;
- quale scope coprono;
- quale domanda possono aiutare a risolvere.

TASK: trovare la procedura applicabile

L1:

PROCESS REGISTER

↓

PROC-A — bootstrap

PROC-B — execution

PROC-C — assurance

A questo punto non abbiamo ancora bisogno di aprire tutti e tre.

Abbiamo una mappa.

11.7 Il Retrieval Gate entra in funzione

Prima di aprire una nuova fonte, PROT-005 chiede:

1. quale informazione manca?
2. questo file è probabilmente la fonte più autorevole per quell'informazione?
3. l'informazione è già disponibile in una fonte letta?
4. il task richiede davvero questo livello di dettaglio?

Queste quattro domande costituiscono il **Retrieval Gate**.

Il gate non è un modulo burocratico.

È una disciplina cognitiva.

Ogni nuova lettura deve rispondere alla domanda:

| perché sto aprendo proprio questo file?

11.8 Domanda 1 — Quale informazione manca?

Questa è la domanda più importante.

Se non sappiamo che cosa manca, rischiamo di cercare senza criterio.

S0:

- goal
- scope
- stato

NON SO:

- quale protocollo impone il guard

La prossima lettura deve cercare il protocollo applicabile.

Non serve aprire una decisione storica che racconta perché quel protocollo è nato.

Il retrieval resta legato al gap corrente.

11.9 Il gap deve essere espresso in modo concreto

Una formula come:

| *"mi serve più contesto"*

è troppo vaga.

Meglio:

- mi manca l'authority;
- mi manca lo status corrente;
- mi manca il protocollo applicabile;
- mi manca la source of truth;
- mi manca la ragione storica;
- mi manca l'evidence che supporta il claim.

Più il gap è preciso, più la route può essere precisa.

11.10 Domanda 2 — Qual è la fonte più autorevole?

Una volta identificata l'informazione mancante, bisogna capire dove cercarla.

Non basta chiedere:

| *"quale file parla di questo tema?"*

Bisogna chiedere:

| **"quale fonte dovrebbe avere authority su questo tipo di informazione?"**

In altre parole: quale fonte è autorevole **rispetto a quella domanda**, non in senso assoluto.

Se cerchiamo una regola corrente, una discussione storica è una fonte debole.

Se cerchiamo l'origine di una decisione, quella discussione può diventare rilevante.

L'autorità è quindi legata alla domanda.

11.11 Source precedence in pratica

Senza anticipare il Capitolo 12, possiamo usare una regola semplice.

Per una domanda normativa:

```
GOVERNANCE / CANON
prima di
PROCESS / PROTOCOL
prima di
EVIDENCE / HISTORICAL
```

Per un execution fact:

```
AUTHORITY / CANON
↓
RUNTIME STRUTTURATO
↓
DERIVED STATE / HUMAN VIEW
```

La source precedence evita di iniziare da una fonte semanticamente simile ma gerarchicamente debole.

11.12 Domanda 3 — Ho già quell'informazione?

Questa domanda impedisce il retrieval ridondante.

Supponiamo di aver già letto una fonte autorevole che dice STATUS = ACTIVE.

Aprire altri cinque file solo per ritrovare lo stesso status non aggiunge necessariamente valore.

Prima di leggere ancora:

| l'informazione che cerco è già disponibile con sufficiente authority?

Se sì, il retrieval può fermarsi su quel punto.

11.13 Delta preferred

Questa logica diventa ancora più importante nei follow-up.

Se la baseline è già nota e affidabile, WCM preferisce il delta.

```
BASELINE CONOSCIUTA
+
DELTA NUOVO
↓
AGGIORNA SOLO CIÒ CHE SERVE
```

non:

```
FOLLOW-UP
↓
RILEGGI L'INTERA BASELINE
```

Questo riduce lavoro non necessario senza rinunciare alla possibilità di verificare le fonti quando serve.

11.14 Domanda 4 — Mi serve davvero altro?

Questa è la domanda che chiude il gate.

Un sistema può continuare a cercare indefinitamente.

INDEX-FIRST impone invece una scelta:

| il task richiede davvero un altro livello di dettaglio?

Se la risposta è no, si ferma.

Se la risposta è sì, bisogna sapere perché.

```
CONTESTO ATTUALE:  
- authority chiara  
- processo chiaro  
- status chiaro  
- next step chiaro
```

```
ALTRO RETRIEVAL?  
NO
```

11.15 L2 — Authority / Procedure

L2 è il cuore operativo.

Qui si leggono le fonti necessarie per capire:

- cosa vale;
- cosa è consentito;
- quale procedura si applica;
- quale stato conta;
- quale next transition è corretta;
- quali stop condition esistono.

Le fonti L2 possono essere governance, baseline canonica, processi, protocolli, decisioni frozen, current state, contract specifici o runtime pertinente.

11.16 Un esempio astratto di L2

Task:

| "Posso eseguire questa operazione?"

L1 ha individuato un processo, due protocolli, una decisione storica e una evidence.

Il Retrieval Gate dice che mancano regola corrente e authority applicabile.

Quindi apriamo:

La decisione storica e l'evidence restano chiuse.
Se le due fonti L2 risolvono il task, il retrieval termina.

11.17 L2 non significa sempre molti documenti

A volte basta una sola fonte.

Se la domanda è:

| *"Qual è lo status corrente di questo nodo?"*

e l'indice punta a una source of truth chiaramente definita, può essere sufficiente leggere quella.

INDEX-FIRST non impone un numero minimo di file.

Il criterio è:

| **contesto sufficiente, non quantità minima di documenti.**

11.18 Quando L2 non basta

L2 può lasciare un gap.

Per esempio:

- due fonti autorevoli sembrano confliggere;
- un protocollo cita una decisione non disponibile;
- un claim richiede verifica;
- lo status è chiaro ma il perché è necessario al task;
- una source of truth sembra stale;
- serve ricostruire lineage.

A questo punto può diventare necessario L3.

11.19 L3 — Evidence / Historical / Raw

L3 contiene il contesto profondo.

Può includere evidence, POC, raw data, storico, vecchie decisioni, discussioni, esperimenti e lineage esteso.

L3 non è meno importante.

È **meno necessario come bootstrap standard.**

11.20 Perché non si passa automaticamente a L3

Aprire L3 automaticamente crea tre rischi.

Primo rischio: rumore

Molti dettagli non cambiano il task.

Secondo rischio: storico scambiato per corrente

Una vecchia fonte può essere semanticamente molto simile alla baseline.

Terzo rischio: costo

Tempo, token e attenzione vengono spesi senza una necessità operativa.

Per questo L3 è on demand.

11.21 Quando L3 è necessario

L3 è corretto quando il task richiede:

- audit;
- lineage;
- verifica di un claim;
- ricostruzione causale;
- risoluzione di un conflitto;
- promozione evidence → baseline;
- comprensione di un failure;
- revisione trasversale.

Qui leggere più profondamente non è un anti-pattern.

È la richiesta stessa a giustificarlo.

11.22 Il passaggio L2 → L3 deve essere esplicito

Un buon reasoning può formulare il passaggio così:

```
L2 NON BASTA
perché:
- fonte A e fonte B confliggono
```

```
MI MANCA:
- lineage della decisione
```

PROSSIMA FONTE:
- decision record / evidence collegata

Questa esplicitazione mantiene il retrieval tracciabile.

11.23 Stop When Sufficient

Il retrieval deve fermarsi quando il contesto è sufficiente.

Ma "sufficiente" non significa:

| *"ho trovato qualcosa che sembra giusto."*

Significa:

| **ho le informazioni necessarie per compiere correttamente il task, con authority e rischio adeguatamente risolti.**

11.24 Il Context Sufficiency Gate

PROC-005 fornisce una checklist utile.

Il contesto è sufficiente quando l'attore sa, per quanto pertinente al task:

- chi è;
- qual è progetto/goal;
- se esiste un workflow da riprendere;
- cosa è già affidabile;
- quale fonte persistente è autorevole;
- quale authority possiede;
- quale scope è autorizzato;
- quale transizione viene dopo;
- quali processi/protocolli si applicano;
- quali azioni richiedono escalation;
- quale true stop condition deve raggiungere.

Non ogni domanda richiede la stessa profondità.

Ma se un elemento necessario manca, il retrieval deve continuare.

11.25 Sufficienza proporzionata al rischio

Una domanda descrittiva può richiedere poco.

Un'operazione materiale o persistente può richiedere molto di più.

DOMANDA INFORMATIVA
→ status + fonte corrente
→ STOP

OPERAZIONE MATERIALE

→ authority + scope + procedure + expected state + guard

→ STOP

La soglia di sufficienza dipende dal rischio.

11.26 Quando il retrieval deve fermarsi perché c'è un problema

Stop When Sufficient non è l'unico motivo per fermarsi.

Possiamo anche fermarci perché il contesto rivela una condizione bloccante.

DUE FONTI AUTOREVOLI IN CONFLITTO

↓

NO SILENT CONFLICT RESOLUTION

↓

STOP / ESCALATE

In questo caso non abbiamo abbastanza per eseguire.

Abbiamo abbastanza per sapere che **non possiamo eseguire correttamente**.

11.27 Knowledge Trust Gate durante INDEX-FIRST

Se un indice o una route appaiono stale, il sistema non deve fidarsi ciecamente della mappa.

Segnali:

- index incoerente con baseline nota;
- relazione critica BROKEN;
- Knowledge Health non affidabile su un percorso sensibile;
- runtime e human view divergono.

In questi casi:

NAVIGATION PROBLEM

↓

ASSURANCE / RECONCILIATION

↓

poi continua il task

INDEX-FIRST presuppone una mappa sufficientemente sana.

11.28 Esempio completo: trovare una regola corrente

Task:

| *"Qual è la regola corrente che governa questa operazione?"*

L0

Capisco che la domanda riguarda una regola del metodo.

L1

Method KB / Process Register.

Trovo protocollo current e concept storico.

Retrieval Gate

Manca: regola normativa corrente.

Fonte più autorevole: protocollo current.

L2

Leggo il protocollo.

La regola è chiara.

Stop

Non apro il concept storico.

11.29 Esempio completo: ricostruire perché una regola esiste

Task:

| *"Perché questa regola è stata introdotta?"*

L0

La domanda è storica/causale.

L1

Indice delle decisioni / evidence.

L2

Leggo la decisione corrente per identificare il lineage.

Gap

So cosa vale, ma non ancora perché è nato.

L3

Apro evidence / decision record precedente.

Stop

Il lineage è sufficiente.

Qui L3 era necessario.

11.30 Esempio completo: riprendere un workflow

Task:

| *"Continua il lavoro."*

L0

Controllo Working Memory e runtime workflow.

Trovo STATUS = INTERRUPTED_RESUMABLE e NEXT_TRANSITION = REVIEW.

Resume Priority

Non cerco un nuovo task.

L2

Leggo solo authority, scope e fonti necessarie alla REVIEW.

Stop

Quando il Context Sufficiency Gate è verde, riprendo da REVIEW.

INDEX-FIRST serve quindi anche a riprendere il punto giusto del lavoro.

11.31 Esempio completo: fonte stale

Task:

| *"Qual è lo stato corrente?"*

L'indice porta a una human view che dice ACTIVE.

Il runtime strutturato dice COMPLETED.

Per execution facts il runtime prevale.

Non si media.

Si applica reconciliation.

Dopo riallineamento, il retrieval può usare la vista corretta.

11.32 Anti-pattern: ricerca per parola e basta

```
CERCO "approval"  
↓  
APRO I PRIMI RISULTATI  
↓  
SINTETIZZO
```

Problema:

- nessuna authority;
- nessuno status;
- nessuno scope;
- storico e current possono mescolarsi.

La ricerca può aiutare.

Ma deve stare dentro una route governata.

11.33 Anti-pattern: full reload preventivo

| *"Prima di rispondere, leggo tutta la KB."*

Sembra prudente.

Ma viola task scope e progressive disclosure.

La forma corretta è:

```
TASK  
↓  
INDEX  
↓  
FONTI MINIME  
↓  
APPROFONDISCI SOLO SE MANCA QUALCOSA
```

11.34 Anti-pattern: fermarsi alla prima fonte

Pattern opposto:

```
TROVO UNA FONTE  
↓  
STOP
```

senza verificare status, authority, scope e precedence.

INDEX-FIRST non significa minimalismo cieco.

Significa **minimo sufficiente**.

11.35 Anti-pattern: leggere lo storico prima della baseline

Una fonte storica può essere molto ricca.

Ma se il task è corrente, partire da lì può distorcere il contesto.

Meglio:

```
CURRENT FIRST
↓
HISTORY ON DEMAND
```

11.36 Anti-pattern: non dichiarare il gap

Se il reasoning continua a leggere senza sapere che cosa sta cercando, il retrieval può espandersi senza controllo.

Una buona disciplina è:

```
MI MANCA X
↓
LA FONTE MIGLIORE È Y
↓
APRO Y
```

Questo rende il percorso spiegabile.

11.37 INDEX-FIRST non è una ricetta rigida

Il pattern L0-L3 è una struttura.

Non un algoritmo che vieta ogni deviazione.

Un audit, una migrazione o una ricerca di contraddizioni trasversali possono richiedere letture ampie.

Un task operativo può richiedere runtime prima di un indice documentale.

Un follow-up può usare delta retrieval.

La regola costante è:

| ogni espansione del contesto deve avere una ragione legata al task.

11.38 INDEX-FIRST e determinismo

INDEX-FIRST non rende deterministico il reasoning.

Riduce però alcune fonti di variabilità.

Per esempio stabilisce:

- dove iniziare;
- quali livelli esistono;
- quale precedenza considerare;
- quando approfondire;
- quando fermarsi.

Questo rende il percorso di retrieval più ripetibile.

Non rende deterministica l'interpretazione semantica: due agenti possono ancora leggere la stessa fonte e formulare sfumature diverse. INDEX-FIRST riduce la variabilità del percorso, non elimina la cognition.

11.39 INDEX-FIRST e costi

Il protocollo nasce anche per ridurre token, latenza, letture irrilevanti, tempo di bootstrap e rischio di contraddizione.

Ma questi sono effetti misurabili da validare nel tempo.

La baseline corrente non pretende di aver dimostrato una percentuale universale di risparmio.

Il principio architetturale viene prima della metrica.

11.40 INDEX-FIRST come comportamento quotidiano

Possiamo condensare tutto in sei domande:

1. DOVE SONO?
2. CHE COSA MI MANCA?
3. QUAL È LA MAPPA GIUSTA?
4. QUAL È LA FONTE PIÙ AUTOREVOLE?
5. MI SERVE DAVVERO SCENDERE PIÙ IN PROFONDITÀ?
6. HO ABBASTANZA PER AGIRE CORRETTAMENTE?

Se queste domande diventano automatiche, INDEX-FIRST smette di essere una tecnica speciale.

Diventa il modo normale di usare la memoria organizzativa.

11.41 Dove siamo arrivati

1. INDEX-FIRST è una disciplina di retrieval, non il semplice gesto di aprire un indice.
2. L0 orienta.
3. L1 mostra la mappa.
4. L2 apre authority e procedure necessarie.
5. L3 apre evidence, storico e raw solo quando il task lo richiede.

6. Il Retrieval Gate obbliga a dichiarare il gap prima di espandere il contesto.
7. Source precedence aiuta a scegliere quale fonte aprire per prima.
8. Delta preferred evita di ricostruire tutto nei follow-up.
9. Stop When Sufficient limita il retrieval quando il contesto è adeguato.
10. Conflitti autorevoli, drift o trust failure possono imporre uno stop/escalation invece dell'esecuzione.
11. INDEX-FIRST riduce lo spazio di ricerca senza cancellare la profondità della memoria.

Nel Capitolo 10 abbiamo visto la mappa.

In questo capitolo abbiamo imparato a percorrerla.

Nel prossimo entreremo nel problema più delicato:

che cosa succede quando due informazioni parlano dello stesso tema ma non hanno lo stesso peso?

La risposta è la **Source Precedence**.

Frozen Source Map — 11

Fonti canoniche principali usate:

- WCM_AGENT_START.md — bootstrap, Working Memory, Resume Priority, Knowledge Trust Gate, source precedence e stop condition;
- wcm/process-book/protocols/PROT-005_INDEX_FIRST_PROGRESSIVE_RETRIEVAL.md — L0-L3, Retrieval Gate, task scope, progressive disclosure, delta preferred, no silent conflict resolution e Stop When Sufficient;
- wcm/process-book/processes/PROC-005_AGENT_READY_CONTEXT_BOOTSTRAP.md — Context Sufficiency Gate, Resume Priority e bootstrap progressivo;
- wcm/kb/concepts/CONCEPT-007_AGENT_READY_KNOWLEDGE_ARCHITECTURE.md — Knowledge Navigation Layer e definizione Agent-Ready;
- wcm/documentation/process-memory-book/chapters/10_knowledge_navigation_layer.md — continuità pedagogica con la mappa del layer;
- FIG-005_WCM_KNOWLEDGE_NAVIGATION_LAYER.svg — figura già approvata nel Capitolo 10, richiamata concettualmente ma non duplicata.

Review Closure

- Technical Review — PASS dopo micro-correzioni;
- Human Comprehension Review — PASS dopo micro-correzioni;
- L0-L3 = progressive retrieval, non sequenza rigida obbligatoria — verified;
- L3 = deep context on demand — verified;
- Retrieval Gate coerente con PROT-005 — verified;
- source authority qualificata rispetto alla specifica informazione — verified;
- runtime precedence limitata agli execution facts — verified;
- Memory is not authority — verified;
- Delta Preferred — verified;

- Resume Priority correttamente collocata nel bootstrap — verified;
- Stop When Sufficient $\neq$ prima risposta plausibile — verified;
- stop/escalation $\neq$ sufficiency — verified;
- INDEX-FIRST aumenta la ripetibilità del retrieval, non rende deterministica la semantica — verified;
- nessun claim quantitativo universale — verified;
- scope generale / nessun riferimento project-specific — PASS;
- nuova figura — NOT REQUIRED / FIG-005 del Capitolo 10 sufficiente come mappa del layer.

Freeze verdict: CHAPTER 11 FROZEN — 2026-08-29.